

Iteradores

default

Você raramente precisa escrever um iterador do zero: o Python já traz vários prontos.

Listas, strings, tuplas, dicionários e conjuntos entregam iteradores com `iter()` ;

`enumerate` , `zip` , `reversed` e `map` devolvem iteradores preguiçosos; e o módulo `itertools` tem ferramentas para contar, repetir e encadear. Nesta aula: os iteradores que já vêm com a linguagem e quando cada um se esgota.

`iter()` nos tipos padrão

`enumerate` · `zip` · `reversed`

`range`

`dict.items()`

`iter(função, sentinela)`

`itertools`

Formação Claude Code 2026 · Baseado em docs.python.org (Built-in Functions: `iter`, `enumerate`, `zip`, `reversed`, `range`; Built-in Types: dictionary views; `itertools`), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em [exemplos/](#) · Saídas reais

Cada coleção tem *o seu iterador*

embutidos.py

```
for valor in ["ab", (1, 2), {"x": 1}, {7}]:  
    it = iter(valor)  
    print(type(it).__name__, "->", next(it))
```

SAÍDA

```
str_ascii_iterator -> a  
tuple_iterator -> 1  
dict_keyiterator -> x  
set_iterator -> 7
```

- ✓ String, tupla, dicionário e set são **iteráveis**: `iter()` devolve um iterador próprio de cada tipo
- ✓ O iterador de dicionário entrega as **chaves**
- ✓ O set não garante ordem: não conte com a posição dos itens

Os nomes (`tuple_iterator` , `set_iterator` ...) são detalhes internos: o que importa é que todos respondem a `next()` .

enumerate, zip e reversed *não são listas*

preguicosos.py

```
nums = [3, 1, 2]
pares = enumerate(nums)
print(pares)           # um objeto, não uma lista
print(list(pares))     # materializa os itens
print(list(pares))     # e ele já se esgotou
print(list(reversed(nums)))
print(list(zip("abc", nums)))
```

SAÍDA

```
<enumerate object at 0x792435926160>
[(0, 3), (1, 1), (2, 2)]
[]
[2, 1, 3]
[('a', 3), ('b', 1), ('c', 2)]
```

- ✓ Eles devolvem **iteradores**: calculam cada item só quando você pede
- ✓ Imprimir mostra só o objeto; `list()` materializa os itens
- ✓ Como todo iterador, **se esgotam** depois de uma volta

O endereço `0x...` muda a cada execução: é só a posição do objeto na memória.

range: iterável *que não se esgota*

range_reusa.py

```
r = range(0, 10, 3)
print(list(r))
print(list(r))          # range é iterável: recomeça
print(6 in r, 7 in r)
print(len(r), r[-1])
```

SAÍDA

```
[0, 3, 6, 9]
[0, 3, 6, 9]
True False
4 9
```

- ✓ `range` é uma **sequência**, não um iterador: percorre quantas vezes quiser
- ✓ Não guarda os números: calcula na hora, então `range(10**9)` ocupa pouca memória
- ✓ Aceita `in`, `len()` e índice, como uma lista

Chaves, valores *e pares*

dicionario.py

```
estoque = {"maçã": 4, "uva": 0, "kiwi": 7}
print(list(estoque))           # chaves
print(list(estoque.values()))
for fruta, qtd in estoque.items():
    if qtd == 0:
        print(fruta, "esgotou")
```

SAÍDA

```
['maçã', 'uva', 'kiwi']
[4, 0, 7]
uva esgotou
```

VOCÊ ESCRIVE

RECEBE A CADA VOLTA

`for k in d`

a chave

`d.keys()`

a chave

`d.values()`

o valor

`d.items()`

a tupla `(chave, valor)`

São **views**: refletem o dicionário atual. Não altere o tamanho do dicionário enquanto percorre um deles.

iter(função, *sentinela*)

iter_sentinela.py

```
from functools import partial
import io

texto = io.StringIO("abcdefgh")
ler4 = partial(texto.read, 4)

# iter(função, sentinela): chama até receber ""
for bloco in iter(ler4, ""):
    print(bloco)
```

SAÍDA

```
abcd
efgh
```

- ✓ Com dois argumentos, `iter()` chama a função sem argumentos até ela devolver o valor **sentinela**
- ✓ `partial(texto.read, 4)` fixa o argumento: cada chamada lê 4 caracteres
- ✓ É o jeito clássico de ler um arquivo em blocos de tamanho fixo

`io.StringIO` faz uma string se comportar como arquivo. Num arquivo real, a ideia é a mesma.

Contar, repetir *e encadear*

ferramentas.py

```
from itertools import count, cycle, islice, chain

print(list(islice(count(10, 5), 4)))      # 10, 15, 20, 25
print(list(islice(cycle("AB"), 5)))      # A B A B A
print(list(chain([1, 2], (3,), "45")))    # junta iteráveis
```

SAÍDA

```
[10, 15, 20, 25]
['A', 'B', 'A', 'B', 'A']
[1, 2, 3, '4', '5']
```

FERRAMENTA

O QUE FAZ

`count(início, passo)`

conta sem fim

`cycle(iterável)`

repete o iterável sem fim

`islice(it, n)`

corta os primeiros `n` itens

`chain(a, b, ...)`

percorre um iterável depois do outro

`count` e `cycle` são infinitos: nunca faça `list(count())` sem limitar com `islice`.

Escala de *turnos*

Enunciado: distribua os dias da semana entre três pessoas, em rodízio, sem usar índices nem `%`.

Uma resposta possível

```
from itertools import cycle

pessoas = ["Ana", "Bia", "Caio"]
dias = ["seg", "ter", "qua", "qui", "sex"]

for dia, pessoa in zip(dias, cycle(pessoas)):
    print(f"{dia}: {pessoa}")
```

SAÍDA

```
seg: Ana
ter: Bia
qua: Caio
qui: Ana
sex: Bia
```

- ✓ `cycle(pessoas)` é infinito, mas o `zip` para no menor iterável (os dias)
- ✓ Desafio: numere os dias com `enumerate(..., start=1)`

Teste o que *ficou*

1. `next(iter({"a": 1}))` devolve:

A) 1

B) "a"

C) ("a", 1)

D) Erro

B. Dicionário itera pelas chaves.

2. Qual destes dá para percorrer várias vezes?

A) `zip(a, b)`

B) `enumerate(a)`

C) `range(5)`

D) `reversed(a)`

C.

3. `list(islice(count(1), 3))` vale:

A) [0, 1, 2]

B) [1, 2, 3]

C) [1, 1, 1]

D) Nunca termina

B.

4. Para ter (chave, valor) num for sobre um dicionário d:

A) `for k, v in d`

B) `for k, v in d.items()`

C) `for k, v in d.values()`

D) `for k, v in iter(d)`

B.

Iteradores prontos *para usar*

As coleções do Python já entregam iteradores com `iter()`, `enumerate`, `zip` e `reversed` são iteradores preguiçosos e se esgotam; `range` e as views do dicionário não. `iter(função, sentinela)` repete uma chamada até um valor de parada, e o `itertools` conta, repete e encadeia sem montar listas.

`iter()`

`enumerate` · `zip` · `reversed`

`range`

`items()`

`itertools`

Próxima aula → 03 · Usando um Gerador

Fontes: docs.python.org/3/library/functions.html (`iter`, `enumerate`, `zip`, `reversed`, `range`); docs.python.org/3/library/stdtypes.html#dictionary-view-objects; docs.python.org/3/library/itertools.html; docs.python.org/3/library/functools.html#functools.partial; docs.python.org/3/library/io.html#io.StringIO, consultados em outubro de 2026. Exemplos em [exemplos/](#), executados com Python 3.14.4.